

RAGNexus - Hybrid RAG Chatbot

-Vedant Shimpi

1. Project Overview & Chatbot Objective

RAGNexus is a full-stack, AI-powered chatbot built on a Retrieval-Augmented Generation (RAG) architecture. Unlike traditional chatbots that reply from a fixed script or a generic language model, RAGNexus grounds every answer in documents that the user explicitly provides URLs, PDFs, TXT files, or CSVs. This ensures responses are factual, traceable, and accompanied by source citations.

The system combines a FastAPI backend, a React + Vite frontend, a Qdrant vector database for semantic search, a BM25 keyword retriever, a cross-encoder reranker, and the Groq LLM (llama-3.1-8b-instant) for answer generation. When no document is loaded, the chatbot falls back to a general-purpose conversational mode.

Core Objective:

- Allow users to upload or link any document (PDF, URL, TXT, CSV)
 - Answer natural-language questions strictly from ingested content
 - Return answers with cited source chunks for full transparency
 - Maintain persistent chat sessions and conversation history
 - Provide a general fallback chat mode when no document is loaded
-

2. Target Users & Use Case

RAGNexus is designed as a general-purpose intelligent document assistant. Its primary target audiences are:

Students & Researchers Upload research papers, textbooks, or lecture notes and ask direct questions without reading the entire document. Ideal for summarising key findings or cross-referencing multiple sources.

Customer Support Teams Ingest product manuals, FAQs, or policy documents. The chatbot acts as a first-line support agent that retrieves precise answers from company documentation.

Legal & Compliance Professionals Upload contracts, regulations, or guidelines and query them for specific clauses, obligations, or definitions.

Campus Assistants Ingest handbooks, timetables, or notice boards and allow students to query course schedules, hostel rules, exam dates, and more.

3. System Architecture

RAGNexus is structured around two main data flows:

Document Ingestion Flow:

Source (URL / PDF / TXT / CSV)

- Loader (fetches & parses content)
- Chunker (splits into overlapping chunks)
- Embedder (sentence-transformers model)
- PostgreSQL (document & chunk metadata storage)
- Qdrant (dense vector storage)

Question-Answering Flow:

User Question

- HyDE Expansion (generate hypothetical answer for better embedding)
- Hybrid Retriever (Qdrant dense search + BM25 keyword search)
- Cross-Encoder Reranker (re-scores retrieved chunks)
- Top-K Contexts selected
- Groq LLM (llama-3.1-8b-instant) generates grounded answer
- Answer + Source Citations returned to frontend

4. Conversation Flow Design

The chatbot follows a structured conversation flow across two modes: Document Mode (RAG-grounded) and Basic Chat Mode (general LLM).

Step 1 — Welcome & Onboarding The frontend greets the user, presents the option to upload a document or start a chat session, and creates a unique session ID stored in PostgreSQL.

Step 2 — Document Ingestion (optional) The user pastes a URL or uploads a file. The backend ingests, chunks, embeds, and confirms success with a `document_id`. The UI displays the document in the workspace.

Step 3 — User Query The user types a natural-language question in the chat input. The frontend sends the question along with the `session_id` and (optionally) `document_id` to the `/ask` endpoint.

Step 4 — Retrieval & Answer Generation The backend runs the HyDE → hybrid retrieval → rerank → Groq generation pipeline and returns a grounded answer with numbered source citations.

Step 5 — Follow-up & History The conversation continues with full history persisted per session. The user can ask follow-up questions referencing prior context.

Step 6 — Fallback (No Document) If no `document_id` is provided or the query falls outside retrieved context, the system routes to `/chat/basic` which calls Groq directly without retrieval constraints.

5. Fallback & Error Handling

RAGNexus implements multi-level fallback and error handling to ensure the chatbot always responds gracefully.

No Document Loaded When a user asks a question without providing a `document_id`, the request is automatically routed to `POST /chat/basic`, which calls Groq directly for a general answer.

Context Not Found in Document If the hybrid retriever returns no relevant chunks above the confidence threshold, the system responds: *"I*

could not find relevant information in the provided document for your question. Please try rephrasing, or use basic chat mode for general queries."

Invalid File Type The /ingest/file endpoint validates file extensions. If an unsupported type is uploaded, the API returns HTTP 422 with the message: *"Only PDF, TXT, and CSV files are supported."*

Qdrant / Database Unavailable If the vector store or PostgreSQL is unreachable, the backend returns HTTP 503 with a descriptive error message and logs the exception for debugging.

Malformed / Empty Input Empty queries or missing required fields trigger Pydantic validation errors returned as HTTP 422 with field-level error descriptions.

6. Tech Stack & Setup

Component	Technology
Backend Framework	FastAPI + SQLAlchemy (Python 3.10+)
Frontend	React + Vite (Node.js 18+)
Vector Database	Qdrant (Docker)
Relational Database	PostgreSQL / Supabase
LLM Provider	Groq (llama-3.1-8b-instant)
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
Keyword Retrieval	BM25 (rank_bm25)
Reranker	Cross-Encoder (sentence-transformers)

Evaluation (optional)	RAGAs
--------------------------	-------

Screenshots :





